CprE 4870/5870
Lab 7: Custom Hardware Accelerator

*"The only phrase I've ever disliked is, 'Why, we've always done it that way.' I always tell young people, 'Go ahead and do it. You can always apologize later." –**Grace Hopper***

1 Learning Objectives 🎯

By the end of this lab you should be able to

- Decompose a the convolution operation into smaller computations.
- Design a hardware accelerator to accelerate convolution.
- Implement a hardware accelerator and benchmark it against software implementations.

2 Pre-lab ✅

2.1 Lab Checklist

- Complete all previous labs (1-5).
- Access and download the lab data and templates. This can be accessed by the links provided on the Canvas assignment

2.2 Introduction

In this lab, we are going to be designing and implementing a more advanced hardware accelerator for the Zedboard to accelerate our model. You should have concluded in your previous lab exercises that the convolution layers dominate the run-time and energy of your model inferences. By designing and implementing a custom accelerator, it should be possible to achieve improvements in power, performance, or area. This accelerator will compute a single convolutional layer (with built-in 2x2 max-pooling of the outputs). Since the same accelerator can accelerate a dense layer as well, the entire model except for the final softmax layer can be computed in the accelerator.

This lab should realistically take your group **three-four weeks** to complete. A few important suggestions:

- Read through the ENTIRE lab document prior to beginning – this is not a step-by-step walk-through. Rather you are given the building blocks to successfully build a basic accelerator.
- Before designing/implementing an extensive accelerator design, test to basic functionality of the provided template. E.g., verify with simple data the CDMA and control register interface. This will save you significant time debugging and redesigning your processor.
- More critically than prior labs, create a complete high-level design diagram for the overall accelerator and the major submodules PRIOR to writing any HDL.

- Use testbenches to test modules prior to integrating, including the your high-level design prior to synthesizing to integrate with C++. You may see a testbench example for one implementation of the full accelerator in the `tb` directory (you may need to modify for your own accelerator design).

3 Accelerator Design 📐

During this process, the planning and design phase will be the most crucial. Prior to any implementation, your group should have your accelerator design fully architected and described. This whole process should be described in your lab document. This lab is intended to be very open ended to push/allow you to design every operation within your accelerator architecture. Your architecture design decisions should be fully motivated by your exploration during the design phase of this lab and the tests conducted during previous labs. You are free to utilize the designs of your previous labs but this is not a requirement. **A template architecture is described below (see Section 4) and provided to you in the repository. While you are encouraged to use the template, you may create your own or redesign it if you wish. If you use the template, you should return to answering the questions in this section after exploring the template.**

3.1 High-level Architecture

Begin by exploring a convolutional operation. Select a convolutional layer from our model for analysis and use that for all further calculations in this lab. How many operations are required to calculate that layer? How much data is required for those operations (per operation & total)?

Consider a basic software implementation of a basic convolutional operation. Make an assumption about how many cycles this basic operation would take. Make an assumption about memory access time from memory. Describe these assumptions and provide your reasoning/source behind them. Calculate if a CPU based implementation would be computation or memory bound. Include your calculations and your bytes/computation value. We will use these values to compare your hardware design against.

Note that your accelerator design is dependent on both the physical resources available on the FPGA (e.g., BRAM, LUTs, and DSP blocks) and the model itself (e.g., filter and feature map sizes/shapes, quantization amenability, and sparsity). In particular, the number of PEs, interconnection, and memory size must all be determined. You may choose a number of PEs based on filter size (likely easiest to implement, test, etc.) or you may choose to max the number based on DSP blocks, etc. of the FPGA (likely maximize performance). Assume you will double-buffer the input and output activations in on-chip SRAM (i.e., PL BRAM). To answer these questions, report answers to the following questions: How can data reuse be leveraged to reduce your layer's memory access overhead? What dataflow will your design support (i.e., input-, output-, weight-, or row-stationary), why? Determine approximately what tile size would be appropriate for your design. Given your tile size, determine the number of PEs needed.

3.2 Visualizing Your Design

Draw a full system diagram of your accelerator design. It should include where data is stored, the general flow of data through the system, and all major components participating in the convolution operation. Draw a micro-architectural diagram of your accelerator. This diagram should include all data and control interfaces of the design, any internal logic and components within your design.

Note: All diagrams should be done digitally. Include a table of all control signals within your design and configuration values. Include where those signals and values originate from and how those affect the operation of your design.

3.3 Theoretical Calculations

Consider the convolutional problem implementation when applied to your design. How much data is required for a single operation of your accelerator? How many cycles does a single operation take? Is your design computational or memory bound? How could you alter your design to reduce these bottlenecks? Describe the throughput and latency of your design, do you expect to beat your software performance?

3.4 Validating your architecture

Once the architecture is designed, it is strongly encouraged to implement the flow in C++. This ensures that any assumptions you have made in quantization logic, etc perform the same as you expect and you can compare the results of the accelerator with your C++ implementation to ensure it is working correctly. This also ensures the weights and biases you have quantized (and probably transposed) will work correctly when used in the accelerator.

4 A Template Implementation 🧪

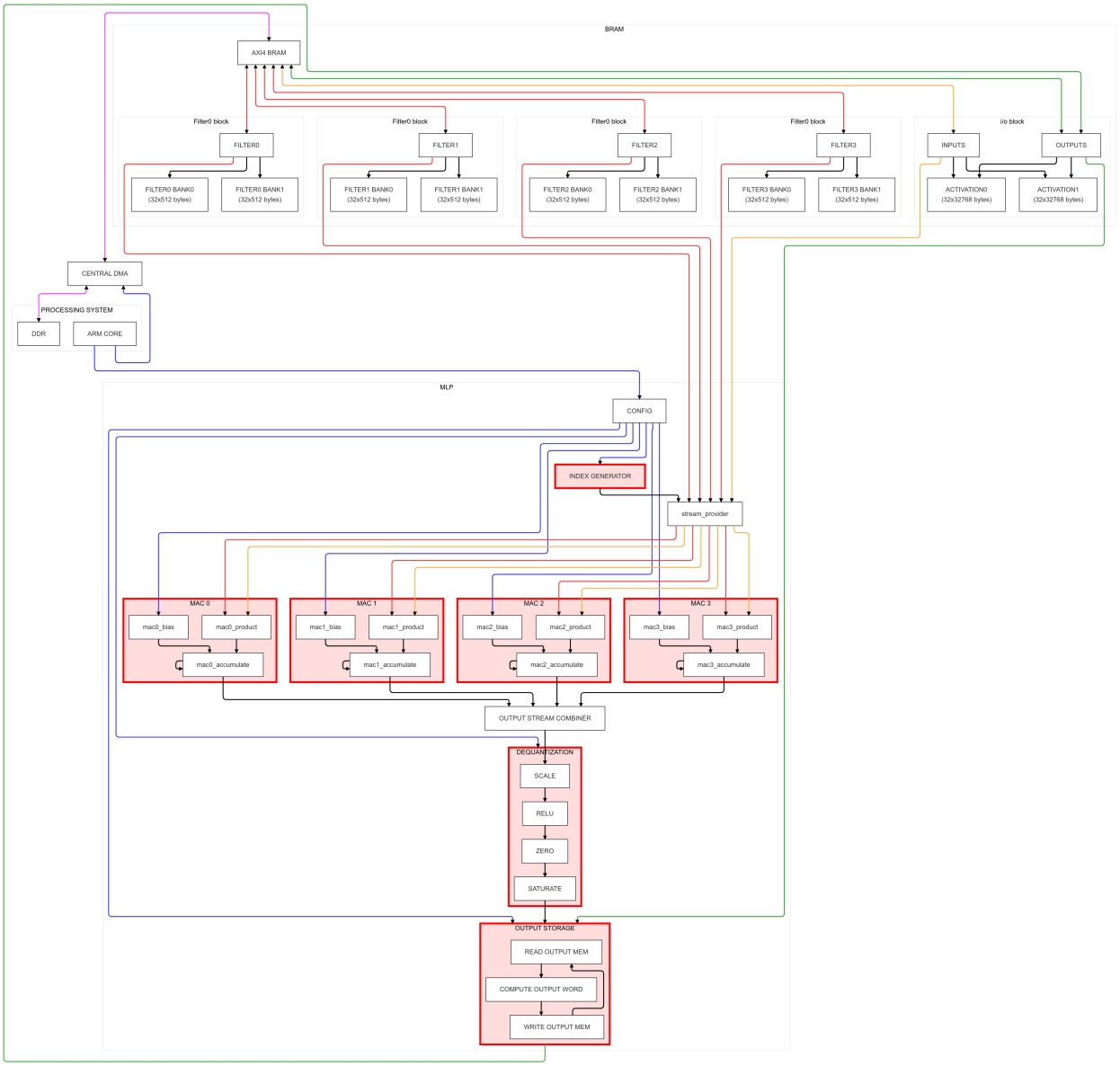


Figure 1: System Architecture Diagram. Red boxes indicate the unimplemented components of the template design that are needed to implement a basic accelerator.

Note: System diagram is available in SVG form next to the template Vivado project.

4.1 Generating the Vivado Project

Use the same steps as before to set up the Vivado template project. From `lab7_template/vivado`, run: `/remote/Xilinx/2020.1/Vivado/2020.1/bin/vivado &`
`source lab7_template.tcl` in Vivado TCL Console

You may get a warning about a missing top-level file. You will have to right-click the block design file in the sources tab and generate a HDL wrapper for it. Right click the generated wrapper and set it as the top module in the design.

This project includes a template block design that is fully functional and intended to allow you to retrieve one 32-bit word of input data and up to four 32-bit words of weight data at a time by using five BRAM interfaces.

4.2 Block Design Overview

4.2.1 BRAM Interface

The BRAM interface is a simple read/write port, but important to note is that data read from the BRAM unit is valid in the cycle following the read command. For example, if you set an address and the enable line high, then the dout line has valid data *after* the next rising edge of the clock. Enter the BRAM sub-diagram to view the BRAM architecture.

There are two banks for storing activations, which is considered input data and output data can be swapped by a configurable pin. This allows you to pass the outputs of the previous layer as inputs to the next layer without moving the activations. There are four double-buffered weight storage elements that allow the accelerator to read from four banks while the alternate buffers are written to by the DMA, then easily swap them out when the acclerator and DMA complete. This allows greater parallelization by allowing the accelerator to run while the DMA copies the weights for the next run from DDR.

4.2.2 Central Direct Memory Access

Finally, the CDMA provides an interface between the DDR memory the processor can access (HP0 port on the Zynq Processing System) and the BRAM modules for the accelerator. **There is no connection from the processor to the BRAM modules, all BRAM accesses *must* be done using the DMA!**

4.3 Accelerator Overview

The accelerator itself is described in a set of VHDL files. It has been designed to use AXI streams as much as possible for consistency and ease of pipelining. This sample accelerator has been designed according to the diagram above, fetching 1 8-bit input and 4 8-bit weights per cycle to continuously feed 4 MAC units. Each MAC unit therefore computes one channel of the output image, with four channels being computed in parallel. This assumes that you have also transposed your weight data so the output channel is the most significant dimension.

4.3.1 AXI-LITE Configuration Registers

The accelerator contains an AXI-LITE configuration bus that allows access from the processor to a set of configuration values in the accelerator such as input size, filter size, etc. See the Address Editor for the base address of the register space. Review `conv_config.vhd` for details, you may add or remove configuration values here as needed to make your accelerator function, but several registers have been provided as examples.

4.3.2 Input & Filter Index Generator

This module generates indices in the BRAM units for the inputs and filter elements. When the convolution starts (`conv_idle` becomes 0), this will continuously provide BRAM indices to the AXI Stream port. You will need to implement this logic yourself. It is suggested to do this using registers, addition, and general logic rather than using another multiply unit.

4.3.3 MAC Stream Provider

This module fetches data from the BRAM units according to the indices passed into it, then passes the BRAM data out for the four MAC units to consume. This module has been written for you to ensure the details of accessing the BRAM units don't become a bottleneck in your debugging. By design, it will fetch the same index from all filter banks, depending on your accelerator design you may wish to modify this unit or make your own.

4.3.4 MAC Units

These are very similar to your Lab 4 MAC units with the exception that TUSER does not exist. Instead, the bias is passed in via a 32-bit bus from the AXI-LITE configuration registers. This allows the MAC units to accept a constant stream of MAC operands. On the first accumulation (after reset or TLAST), the value of the bias bus should be added to the product rather than the value of the accumulation register. For optimal performance, you may want to consider using a pipelined MAC unit.

4.3.5 Output Combiner

As the MAC units only output data rarely, one dequantization stream is sufficient to post-process the outputs of all the MAC units. This module accepts data from the four MAC units to four pipeline registers (to ensure all four MAC units continue accumulating the next output value), then outputs the four in a round-robin fashion, using TID to distinguish the source of each output. This allows for the outputs of all the MAC units to pass through one copy of the dequantization logic, then get sorted later to different locations in the output BRAM.

4.3.6 Dequantization

This unit dequantizes from the 32-bit output of the MAC units to the 8-bit inputs of the next layer. One way of implementing this unit is to analyze the fp32 dequantization and requantization logic to reduce it to a single 32-bit multiply, relu, apply next layer's zero point, and saturation. (Note that the subtraction required to adjust for the current layer's zero point can be added to the bias values prior to running the inference to reduce the amount of computation done at runtime). It is strongly advised to test the entire flow in C++ first, but **especially** this portion. The output of this should be an int8 value which can be directly accepted by the following layer.

For the second dense layer, you may wish to determine a suitable quantization scheme for the outputs, then dequantize from int8 to fp32 in software as part of the softmax operation.

4.3.7 Output Storage

You will first need to generate the output index using the TID and any configuration data necessary. It is suggested to do this using registers, addition, and general logic rather than using another multiply unit.

Because the BRAM interfaces are 32-bit, each 8-bit store requires a read-modify-write operation. You will need to write a small state machine which reads data from the BRAM unit (see MAC Stream Provider for reference), modifies the appropriate 8-bit value, then writes the full 32-bit word back to the BRAM unit. As the MAC units output rarely, it will be sufficient to take 3 or more cycles to store a single output to the BRAM.

4.3.8 Testing

If you wish to implement this template directly, there is a testbench generated by a Python script available. This testbench checks the intermediate AXI stream values and produces output to aid debugging. The behavior of the BRAM units is accurate, so it is also useful for testing that as well. The python source has been provided if you would like to modify the testbench to work with a different acceleration architecture as well. Please use attempt to use this before asking questions about what's wrong with your accelerator outputting incorrect data.

4.4 Running an Inference

Refer to the Lab 4 documentation on exporting your XSA file and running an inference on the zedboard. You will need to double-check timing results to ensure your design is meeting timing, then add pipeline registers as necessary. The template by default is configured to run at 50MHz, but has been known to run at 125MHz, so consider increasing the frequency in the Zynq Processing System prior to measuring speedup. As a side-note for comparison, this accelerator has performed a full model inference in <300ms.

4.4.1 C++ Integration

An MLP.h file has been provided to aid in communicating with the DMA and the accelerator configuration registers. Reminder that **you cannot access the BRAM directly, and must perform all copies through the DMA**. Unfortunately, the DMA does not have cache-coherent access to the processor's memory, so `Xil_DCacheDisable()` is used to disable the processor's data cache. A future improvement would be to connect to the cache-coherent memory port or isolate cache flushes and invalidations to where needed instead of globally disabling the cache, but this is known to work.

Since we wish to keep activations in the accelerator, you will be ignoring dataIn and getOutputData() during the convolutional and dense layers. Instead, assume that the inputs are valid in the accelerator when the layer begins and ensure to flip the activation buffers when the layer ends so the outputs are back in the inputs for the next layer. The softmax layer can then pull from the input BRAM into dataIn as its first operation, then output in getOutputData() as normal. For this to function correctly, you will also have to edit runLayerTest and runInferenceTest to load the input data into the MLP BRAM instead of passing it through dataIn. This removes all need for copying activations between layers.

As the max pooling is performed in hardware now as a part of the convolutional layers, it will likely be easiest to remove the max pooling layers from the model in ML.cpp. In addition, if you refactor your convolutional implementation to not read the output buffer dimensions (instead compute them based on input and weight dimensions) then you can configure the last convolutional layer to output `{2048}` instead of `{4, 4, 128}` and delete the flatten layer as well. This way, your model is now only fused convolutional and dense layers except the final softmax and intermediate activations can stay in the accelerator BRAM.

4.5 Benchmark Your Design

Now it is finally time to benchmark your design. It is assumed that your design is now tested and working when compared to the software implementation running on the ARM CPU. Report the end-to-end latency, throughput, and accuracy of your hardware accelerated implementation. How does this compare to all of the previous implementations (provide all of your data, including from previous labs, in a single table and plot for comparison)? Are you are memory or computationally bound?

For fun and support, post your final fastest HW inference time using your accelerator to the class MS Teams channel for all your class members to compare. Finally, be proud of your design! Publish it to Github and include it on your resume along with your final project.

4.6 Reflection and Final Design

If you have not done so already, fully describe your design in detail. Include any design decisions that were made and why, and a micro-architectural implementation specification (if someone else wanted to, they should be able to replicate your design decisions using only your specification description provided in your lab document). How did you get data into and out of your accelerator? Was this method a bottleneck of the system? What issues did you face? What was easier than you anticipated? What design choices do you regret/would change if you redesigned your implementation? What were you able to ultimately accomplish?

5 Above & Beyond 🚀

- Turn this into a final project and expand on your design:
  - Make an architecture more like a TPU than this simple architecture that uses one BRAM bank per MAC unit.
  - Perform a more complete bottleneck analysis AND improve the memory subsystem (e.g., data orchestration approach, tiles of PEs, etc).
  - Rearchitect your design to leverage sub-8bit quantization. Report effort (latency, energy, area) versus accuracy tradeoffs for various approaches to leveraging quantization. Or add quantization-aware training to a single leveraging of quantization and report the above metrics.
  - Implement multiple accelerator designs and compare their performance.
  - Implement a multi-threaded CPU implementation using the dormant 2nd core of the ARM CPU in the Zedboard PS.

6 Submission Instructions 🗂️

Below is a list of the required turn-in components for the lab to be submitted to Canvas. Only the final submission will be graded, although you are free to resubmit as many times as you please. All submissions must be complete. Even slight changes require an complete re-submission, **NOT** a partial, single file, change. Naming conventions for each file are also provided below.

6.1 Turn-In Checklist

- PDF of your lab document, either from the template or your own format containing all the required lab components (`lab7_report_<Lab Partners Number>.pdf`).
- A zipped archive of your hardware implementation files including source HDL files, control signal spreadsheets, architectural diagrams, and all other relevant information.
- A zipped archive of your software implementation files including C/C++ files and all other relevant information.

6.2 File Structure

```
Canvas
├── lab7_report_<Lab Partners Number>.pdf
├── lab7_hw_<Lab Partners Number>.zip/tgz
│   └── ...
└── lab7_sw_<Lab Partners Number>.zip/tgz
    └── ...
```